Capítulo 24 : Gerenciamento de qualidade

24.1 Qualidade de software

A qualidade de software não é diretamente comparável à qualidade na manufatura.

… pode ser impossível concluir objetivamente se um sistema de software cumpre ou não suas especificações, razões:

1 - é difícil escrever especificações de software completas e precisas

2 - as especificações integram requisitos de várias classes de stakeholders. … podem não incluir os requisitos de todos os grupos de stakeholders.

3 - É impossível medir determinadas características de qualidade diretamente (por exemplo, manutenibilidade), assim, elas não podem ser especificadas de forma não ambígua.

Devido a esses problemas, a avaliação da qualidade de software é um processo subjetivo, em que a equipe de gerenciamento de qualidade precisa usar seu julgamento para decidir se foi alcançado um nível aceitável de qualidade.

Trata-se de responder a perguntas sobre as características do sistema:

1. Durante o processo de desenvolvimento os padrões de programação e documentação foram seguidos?

2. O software foi devidamente testado?

3. O software é suficientemente confiável para ser colocado em uso?

4. O desempenho do software é aceitável para uso normal?

5. O software é útil?

6. O software é bem estruturado e compreensível?

… entendemos que a qualidade de software não implica apenas se a funcionalidade de software foi corretamente implementada, mas também depende dos atributos não funcionais de sistema.

Boehm et al. (1978) sugeriram que havia quinze atributos importantes de qualidade de software, como mostrado na Tabela

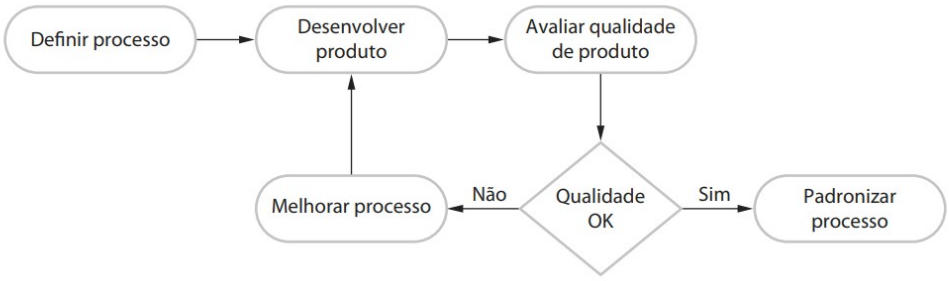
Tabela 24.1 Atributos de qualidade de software

Segurança	Compreensibilidade	Portabilidade
Proteção	Testabilidade	Usabilidade
Confiabilidade	Adaptabilidade	Reusabilidade
Resiliência	Modularidade	Eficiência
Robustez	Complexidade	Capacidade de aprendizado

Um pressuposto do gerenciamento de qualidade de software é que a qualidade do software é diretamente relacionada à qualidade do processo de desenvolvimento de software.

No desenvolvimento de software a relação entre a qualidade de processo e de produto é mais complexa. O desenvolvimento de software é um processo criativo, em vez de mecânico, portanto, a influência de competências e experiências individuais é significativa.

Figura 24.2 Qualidade baseada em processos



asdasdassd

24.2 Padrões de software

Os padrões de software desempenham um papel muito importante no gerenciamento de qualidade de software.

Como parte desse processo de QA, devem ser escolhidas ferramentas e métodos para suportar o uso desses padrões. Uma vez que os padrões foram selecionados para uso, processos específicos de projeto devem ser definidos para monitorar o uso dos padrões e verificar se eles foram seguidos.

Os padrões de software são importantes por três razões:

1. Capturam sabedoria, que é valiosa para a organização.

2. Fornecem um framework para a definição do significado de ‘qualidade’ em uma determinada organização.

3. Ajudam a dar continuidade ao trabalho realizado por uma pessoa, quando retomado e continuado por outra.

Existem dois tipos de padrões de engenharia de software que podem ser definidos e usados no gerenciamento de qualidade de software:

1. Padrões de produto.

Aplicam-se ao produto de software que está sendo desenvolvido.

- Eles incluem padrões de documentos — como a estrutura dos documentos de requisitos;

padrões de documentação — como um cabeçalho de comentário padrão para uma definição de - classe de objeto; e

- padrões de codificação — os quais definem como uma linguagem de programação deve ser usada.

2. Padrões de processo.

Definem os processos que devem ser seguidos durante o desenvolvimento de software.

- Eles devem encapsular as boas práticas de desenvolvimento. Os padrões de processo podem incluir definições de especificação, projeto e processos de validação, ferramentas de suporte do processo e uma descrição dos documentos que devem ser escritos durante esses processos.

Os padrões devem entregar valor, sob a forma de maior qualidade de produto.

Não há razão para definir padrões que sejam caros em termos de tempo e de esforço para serem aplicados e que gerem poucas melhorias na qualidade.

Organismos nacionais e internacionais, como DoD dos Estados Unidos, ANSI, BSI, OTAN e IEEE apoiam a produção de padrões. Tratam-se de padrões gerais que podem ser aplicados em uma gama de projetos.

Organismos como a OTAN e outras organizações de defesa podem exigir que seus próprios padrões sejam usados em seus contratos de desenvolvimento com empresas de software.

Ao usar padrões internacionais como ponto de partida, a equipe de garantia de qualidade deve elaborar um ‘manual’ de padrões. Este deve definir os padrões necessários para sua organização. Exemplos de padrões que poderiam ser incluídos nesse manual são mostrados na Tabela 24.2

… os gerentes de qualidade que definem os padrões devem seguir os seguintes passos:

1. Envolver os engenheiros de software na seleção de padrões de produto.

2. Revisar e modificar regularmente os padrões para refletir mudanças nas tecnologias.

3. Fornecer ferramentas de software para oferecer suporte aos padrões.

Tabela 24.2 Padrões de produto e de processo

Padrões de produto	Padrões de processo
Formulário de revisão de projeto	Condução de revisão de projeto
Estrutura de documento de requisitos	Apresentação do novo código para a construção de sistema
Formato de cabeçalho de método	Processo de versão e *release*
Estilo de programação Java	Processo de aprovação de plano de projeto
Formato de plano de projeto	Processo de controle de mudança
Formulário de solicitação de mudança	Processo de registro de teste

… os gerentes de qualidade que definem os padrões devem seguir os seguintes passos:

1. Envolver os engenheiros de software na seleção de padrões de produto.

2. Revisar e modificar regularmente os padrões para refletir mudanças nas tecnologias.

3. Fornecer ferramentas de software para oferecer suporte aos padrões.

24.2.1 O framework de normas ISO 9001

Existe uma série de normas que podem ser usadas no desenvolvimento de sistemas de gerenciamento de qualidade em todos os setores, chamada ISO 9000.

As normas ISO 9000 podem ser aplicadas a uma variedade de organizações, desde a produção até a indústria de serviços.

A ISO 9001, a mais geral desses padrões, aplica-se a organizações que projetam, desenvolvem e mantêm produtos, incluindo software.

Originalmente, a norma ISO 9001 foi desenvolvida em 1987, com sua mais recente revisão em 2008.

não é propriamente um padrão para o desenvolvimento de software

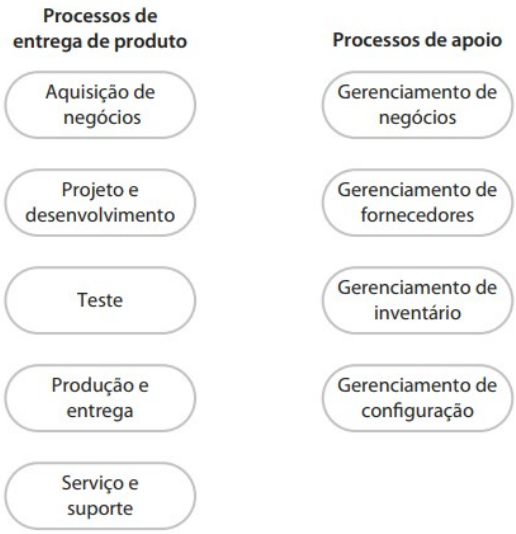
Ela define os princípios gerais da qualidade, descreve os processos gerais de qualidade e estabelece os padrões organizacionais e os procedimentos que devem ser definidos.

Estes devem ser documentados em um manual de qualidade da organização.

As normas ISO 9001 não definem ou prescrevem os processos de qualidade específicos que devem ser usados em uma empresa. Para estar adequada à ISO 9001, uma empresa deve ter definidos os tipos de processos mostrados na Figura 24.3, além de ter procedimentos que demonstrem que seus processos de qualidade estão sendo seguidos.

Isso permite flexibilidade entre setores industriais e tamanhos de empresa.

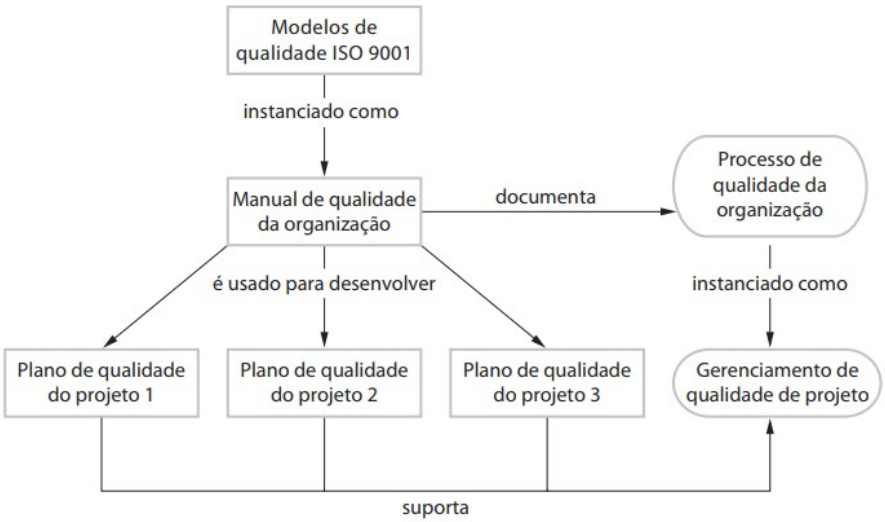
Figura 24.3 Os processos essenciais da ISO 9001



Os relacionamentos entre a ISO 9001, os manuais de qualidade organizacional e os planos individuais de qualidade de projeto são mostrados na Figura 24.4.

Esse diagrama foi obtido a partir de um modelo dado por Ince (1994), que explica como a norma geral ISO 9001 pode ser usada como uma base para processos de gerenciamento de qualidade de software. Bamford e Dielbler (2003) explicaram como a norma posterior ISO 9001:2000, pode ser aplicada em empresas de software

Figura 24.4 ISO 9001 e gerenciamento de qualidade



A norma ISO 9001 assegura que a organização dispõe de procedimentos de gerenciamento de qualidade e segue tais procedimentos. Não há uma garantia de que empresas certificadas com a

ISO 9001 usam as melhores práticas de desenvolvimento de software ou que seus processos gerarão software com alta qualidade

24.3 Revisões e inspeções

Revisões e inspeções são atividades de controle de qualidade que verificam a qualidade dos entregáveis de projeto.

Durante uma revisão, um grupo de pessoas examina o software e a documentação associada à procura de possíveis problemas e não conformidade com padrões.

A revisão deve verificar a consistência e a completude dos documentos ou código em revisão e certificar-se de que os padrões de qualidade foram seguidos.

Elas também são usadas para ajudar a descobrir problemas e omissões no software ou na documentação de projeto.

A revisão é um processo público de detecção de erros, em comparação com processos de testes de componente mais privados.

24.3.1 O processo de revisão

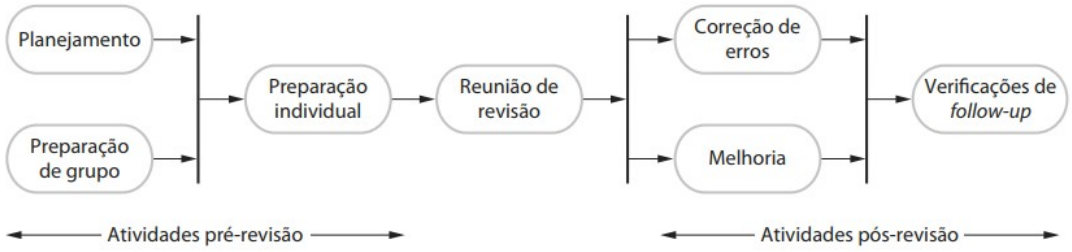
o processo de revisão (Figura 24.5) normalmente é estruturado em três fases:

1. Atividades pré-revisão.

2. Reunião de revisão.

3. Atividades pós-revisão.

Figura 24.5 O processo de revisão de software



24.3.2 Inspeções de programa

As inspeções de programa são ‘revisões em pares’ em que os membros da equipe colaboram para encontrar bugs no programa que está sendo desenvolvido.

Elas complementam os testes, pois não exigem que o programa seja executado. Isso significa que podem ser verificadas versões incompletas do sistema e que representações, tais como modelos UML, podem ser checadas.

A equipe de revisão examina em detalhes os modelos de projeto ou o código de programa e destaca anomalias e problemas para que sejam reparados.

Tabela 24.3 Um *checklist* de inspeção

Classe de defeito	Verificação de inspeção
Defeitos de dados	• Todas as variáveis de programa são iniciadas antes que seus valores sejam usados? • Todas as constantes foram nomeadas? • O limite superior de vetores deve ser igual ao tamanho do vetor ou ao tamanho –1? • Se as *strings* de caracteres são usadas, um delimitador é explicitamente atribuído? • Existe alguma possibilidade de *overflow* de *buffer*?
Defeitos de controle	• Para cada instrução condicional, a condição está correta? • É certo que cada *loop* vai terminar? • As declarações compostas estão posicionadas corretamente entre colchetes? • Em declarações *case*, todos os *cases* possíveis são considerados? • Se um *break* é requerido após cada *case* em declarações *case*, este foi incluído?
Defeitos de entrada/saída	• Todas as variáveis de entrada são usadas? • Todas as variáveis de saída receberam um valor antes de serem emitidas? • Entradas inesperadas podem causar corrupção de dados?
Defeitos de interface	• Todas as chamadas de funções e métodos têm o número correto de parâmetros? • Os parâmetros formais e reais correspondem? • Os parâmetros estão na ordem correta? • Se os componentes acessam memória compartilhada, eles têm o mesmo modelo de estrutura de memória compartilhada?

Algumas possíveis verificações, as quais podem ser feitas durante o processo de inspeção, são mostradas na Tabela 24.3. Gilb e Graham (1993) enfatizam que cada organização deve desenvolver seu próprio checklist de inspeção com base em práticas e padrões locais.

24.4 Medições e métricas de software

A medição de software preocupa-se com a derivação de um valor numérico ou o perfil para um atributo de um componente de software, sistema ou processo.

Comparando esses valores entre si e com os padrões que se aplicam a toda a organização, você pode ser capaz de tirar conclusões sobre a qualidade do software ou avaliar a eficácia dos métodos, das ferramentas e dos processos de software.

Uma métrica de software é uma característica de um sistema de software, documentação de sistema ou processo de desenvolvimento que pode ser objetivamente medido.

Exemplos de métricas incluem:

- o tamanho de um produto em linhas de código;

- o índice Fog (GUNNING, 1962), que é uma medida da legibilidade de uma passagem de texto escrito;

- o número de defeitos relatados em um produto de software entregue, e

- o número de pessoas/dia requerido para desenvolver um componente de sistema.

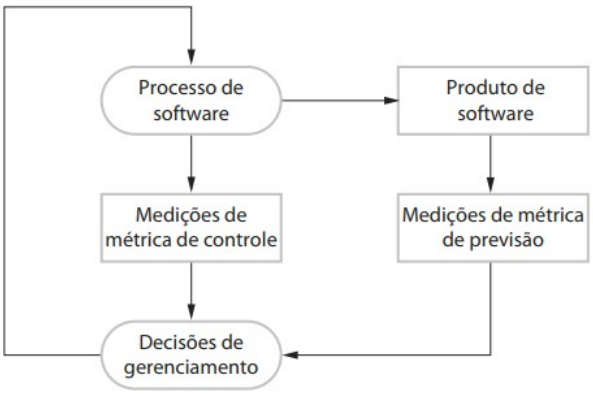
As métricas de software podem ser métricas de controle ou métricas de previsão.

Exemplos de métricas de controle ou de processos são o esforço médio e o tempo necessário para reparar os defeitos relatados.

São exemplos de métricas de previsão: a complexidade ciclomática de um módulo (discutida no Capítulo 8), o comprimento médio dos identificadores em um programa e o número de atributos e operações associadas com as classes de objeto em um projeto.

As métricas de controle e de previsão podem influenciar a tomada de decisão de gerenciamento, como mostrado na Figura 24.6.

Figura 24.6 Medições de previsão e de controle



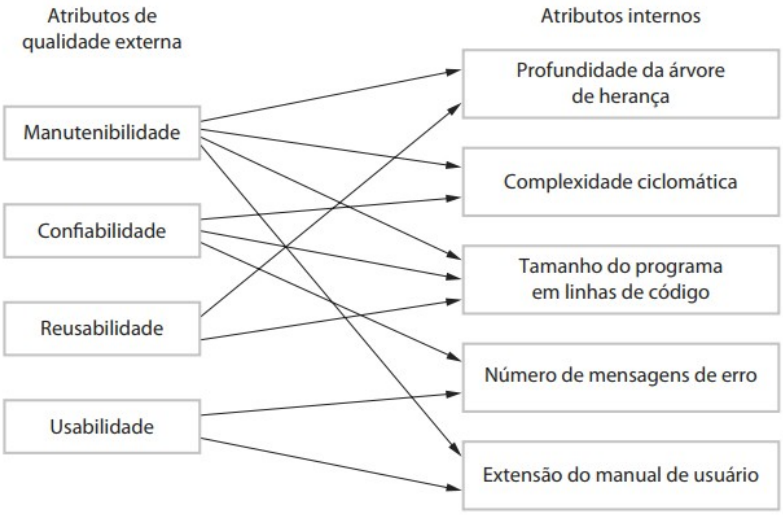
Existem duas maneiras para o uso das medições de um software de sistema:

1. Para atribuir um valor aos atributos de qualidade de sistema.

2. Para identificar os componentes de sistema cuja qualidade não atingiu o padrão.

A Figura 24.7 mostra alguns atributos externos de qualidade dos softwares e atributos internos que poderiam, intuitivamente, ser relacionados a eles.

Figura 24.7 Relacionamentos entre os atributos internos e externos de software



Se a medida do atributo interno deve ser uma previsão útil de características externas de software, três condições devem ser mantidas:

- 1. O atributo interno deve ser medido com precisão. Isso nem sempre é simples e pode exigir ferramentas especiais para fazer as medições.

- 2. Deve existir um relacionamento entre o atributo que pode ser medido e o atributo de qualidade externa de interesse. Ou seja, o valor do atributo de qualidade deve ser relacionado, de alguma forma, com o valor do atributo que pode ser medido.

- 3. Esse relacionamento entre os atributos internos e externos deve ser compreendido, validado e expresso em termos de uma fórmula ou modelo.

Os atributos internos de software, como a complexidade ciclomática de um componente, são medidos com o uso de ferramentas de software que analisam o código-fonte do software.

Existem poucas informações publicamente disponíveis sobre o uso atual da medição sistemática de software na indústria.

24.4.1 Métricas do produto

As métricas de produto são métricas de previsão usadas para medir atributos internos de um sistema de software. O tamanho de sistema, medido em linhas de código, ou o número de métodos associados a cada classe de objeto são exemplos de métricas de produto.

As métricas de produto se dividem em duas classes:

- 1. Métricas dinâmicas, as quais são coletadas por meio de medições efetuadas de um programa em execução.

- 2. Métricas estáticas, que são coletadas por meio de medições feitas de representações do sistema, como o projeto, o programa ou a documentação.

As métricas na Tabela 24.4 são aplicáveis a qualquer programa.

Tabela 24.4 Métricas estáticas de produto de software

Métrica de software	Descrição
Fan-in/Fan-out	*Fan-in* é a medida do número de funções ou métodos que chamam outra função ou método (digamos X). *Fan-out* é o número de funções que são chamadas pela função de X. Um valor alto para *fan-in* significa que X está fortemente acoplado ao resto do projeto e alterações em X terão repercussões extensas. Um valor alto para *fan-out* sugere que a complexidade geral do X pode ser alta por causa da complexidade da lógica de controle necessário para coordenar os componentes chamados.
Comprimento de código	Essa é uma medida do tamanho de um programa. Geralmente, quanto maior o tamanho do código de um componente, mais complexo e sujeito a erros o componente é. O comprimento de código tem mostrado ser uma das métricas mais confiáveis para prever a propensão a erros em componentes.
Complexidade ciclomática	Essa é uma medida da complexidade de controle de um programa. Essa complexidade de controle pode estar relacionada à compreensibilidade de programa. A complexidade ciclomática é discutida no Capítulo 8.
Comprimento de identificadores	Essa é uma medida do comprimento médio dos identificadores (nomes de variáveis, classes, métodos etc.) em um programa. Quanto mais longos os identificadores, mais provável que sejam significativos e, portanto, mais compreensível o programa.
Profundidade de aninhamento condicional	Essa é uma medida da profundidade de aninhamento de declarações *if* em um programa. Declarações *if* profundamente aninhadas são difíceis de entender e potencialmente sujeitas a erros.
Índice *Fog*	Essa é uma medida do comprimento médio de palavras e sentenças em documentos. Quanto maior o valor de um índice *Fog* de um documento, mais difícil a sua compreensão.

A Tabela 24.5 resume o conjunto de Chidamber e Kemerer (1994), às vezes, chamado suite CK, de seis métricas orientadas a objetos.

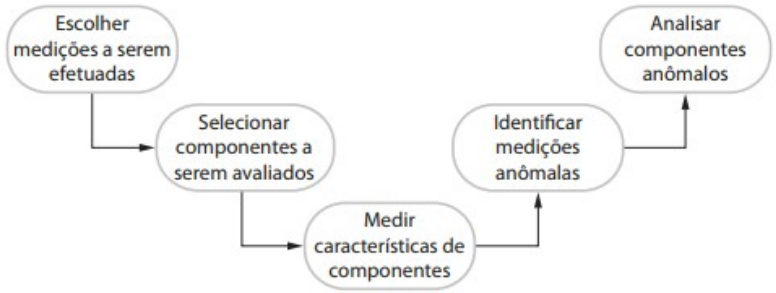
Tabela 24.5 O conjunto de métricas de CK orientadas a objetos

Métrica orientada a objeto	Descrição
Métodos ponderados por classe (WMC, do inglês *weighted methods per class*)	É o número de métodos em cada classe, ponderados pela complexidade de cada método. Portanto, um método simples pode ter uma complexidade de 1 e um método grande e complexo pode ter um valor muito superior. Quanto maior o valor para essa métrica, mais complexa a classe de objeto. Geralmente, os objetos complexos são mais difíceis de compreender. Eles podem não ser logicamente coesos, portanto, não podem ser reusados efetivamente como superclasses em uma árvore de herança.
Árvore de profundidade de herança (DIT, do inglês *depth Of inheritance tree*)	Representa o número de níveis discretos na árvore de herança em que as subclasses herdam atributos e operações (métodos) de superclasses. Quanto mais profunda a árvore de herança, mais complexo o projeto. Muitas classes de objeto podem precisar ser compreendidas para que as classes de objeto nas folhas da árvore sejam entendidas.
Número de filhos (NOC, do inglês *number of children*)	É uma medida do número de subclasses imediatas em uma classe. Ele mede a largura de uma hierarquia de classe, considerando que DIT mede sua profundidade. Um valor alto para NOC pode indicar um maior reúso. Isso pode significar que mais esforço deve ser dispendido na validação de classes de base por causa do número de subclasses que dependem delas.
Acoplamento entre classes de objeto (CBO, do inglês *coupling between object classes*)	Classes são acopladas quando métodos em uma classe usam métodos ou variáveis de instância definidas em uma classe diferente. CBO é uma medida de quanto acoplamento existe. Um valor alto para o CBO significa que as classes são altamente dependentes e, portanto, é mais provável que a mudança em uma classe afete outras classes do programa.
Resposta para uma classe (RFC, do inglês *response for a class*)	RFC é a medida do número de métodos que poderiam ser executados em resposta a uma mensagem recebida por um objeto dessa classe. Mais uma vez, RFC está relacionada com a complexidade. Quanto maior o valor de RFC, mais complexa é a classe e, portanto, mais provável que inclua erros.
Falta de coesão em métodos (LCOM, do inglês *lack of cohesion in methods*)	LCOM é calculada considerando os pares de métodos em uma classe. LCOM é a diferença entre o número de pares de métodos sem atributos compartilhados e o número de pares de métodos com atributos compartilhados. O valor dessa métrica tem sido amplamente discutido, e ele existe em diversas variações. Não está claro se realmente adiciona qualquer informação útil além das que já são fornecidas por outras métricas.

24.4.2 Análise de componentes de software

Um processo de medição que pode ser parte de um processo de avaliação de qualidade de software é mostrado na Figura 24.8.

Figura 24.8 O processo de medição de produto



Os estágios principais nesse processo de medição de componente são:

1. Escolher medições a serem efetuadas

2. Selecionar componentes a serem avaliados.

3. Medir características de componentes.

4. Identificar medições anômalas

5. Analisar componentes anômalos.

Deve-se manter os dados coletados como um recurso organizacional e manter registros históricos de todos os projetos

24.4.3 Ambiguidade de medições

Quando você coleta dados quantitativos sobre software e processos de software, precisa analisar esses dados para entender seu significado.

É fácil interpretar dados erroneamente e fazer inferências incorretas.

No entanto, processos e produtos que estão sendo medidos não são isolados de seu ambiente. O ambiente de negócio está mudando constantemente e é impossível evitar mudanças nas práticas de trabalho só porque eles podem fazer comparações de dados inválidos.

Assim, dados quantitativos sobre as atividades humanas não podem sempre ser tomados pelo valor de face. Muitas vezes, as razões pelas quais um valor medido se altera são ambíguas. Essas razões devem ser pesquisadas em detalhes antes de se tirarem conclusões a respeito de quaisquer medições efetuadas anteriormente.

Capítulo 25 : Gerenciamento de configuração

O gerenciamento de configuração (CM, do inglês Configuration Management) está relacionado com as políticas, processos e ferramentas para gerenciamento de mudanças dos sistemas de software.

Você precisa gerenciar os sistemas em evolução, pois é fácil perder o controle de quais mudanças e versões de componentes foram incorporadas em cada versão de sistema.

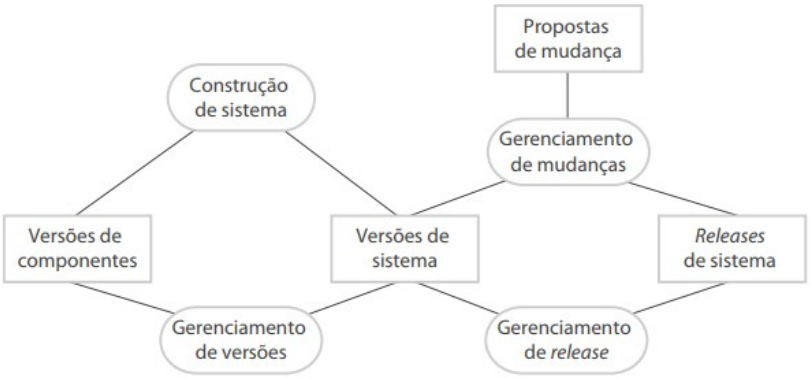
As versões implementam propostas de mudanças, correções de defeitos e adaptações de hardware e sistemas operacionais diferentes.

Pode haver várias versões em desenvolvimento e em uso ao mesmo tempo. Se você não tem procedimentos de gerenciamento de configuração efetivos, você pode desperdiçar esforço modificando a versão errada de um sistema, entregá-la para os clientes ou esquecer onde está armazenado o código-fonte do software para uma versão específica do sistema ou componente.

Pesquisa:

1. O que é Gerenciamento de Configuração (GC)
2. Itens de configuração
3. Controle de versões
4. Gerenciamento de mudanças
5. Ferramentas de GC e sua importância

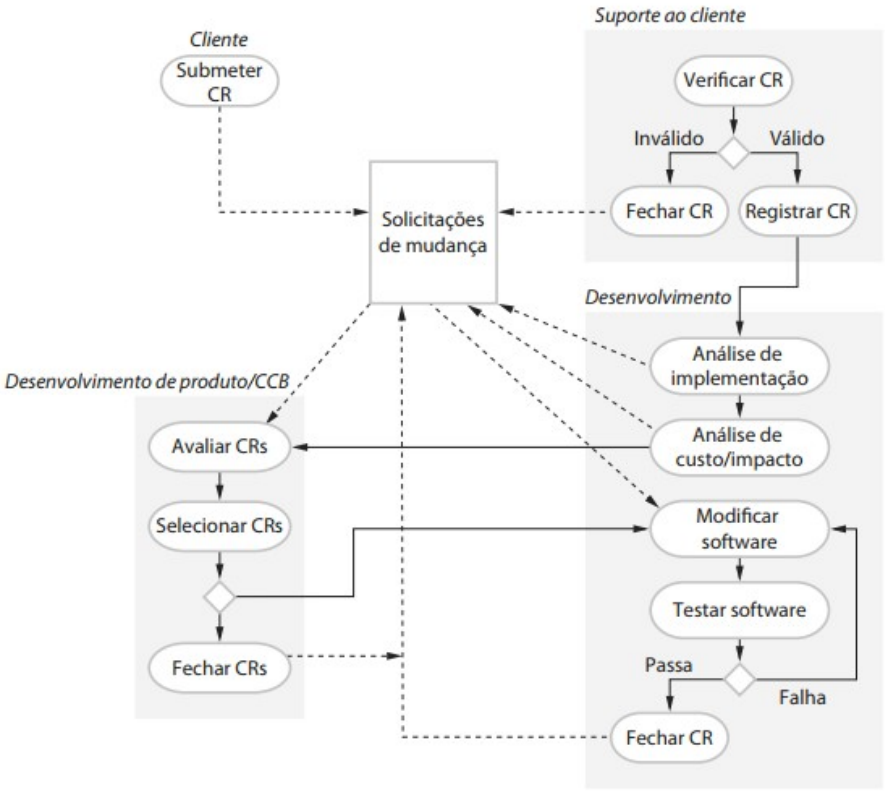
Figura 25.1 Atividades de gerenciamento de configuração



25.1 Gerenciamento de mudanças

A Figura 25.2 é um modelo de um processo de gerenciamento de mudanças que mostra as principais atividades de gerenciamento.

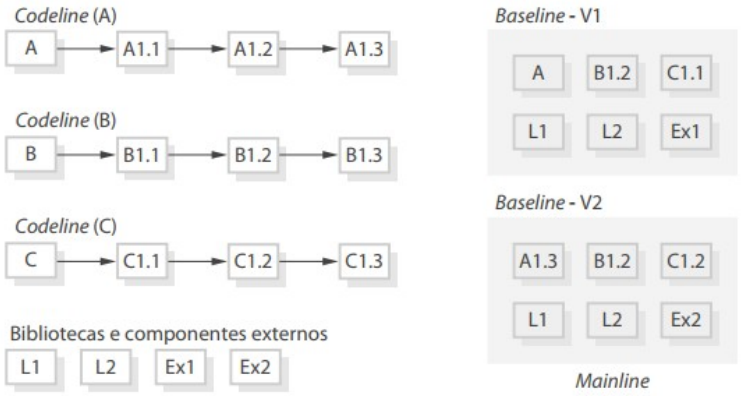
Figura 25.2 O processo de gerenciamento de mudanças



25.2 Gerenciamento de versões

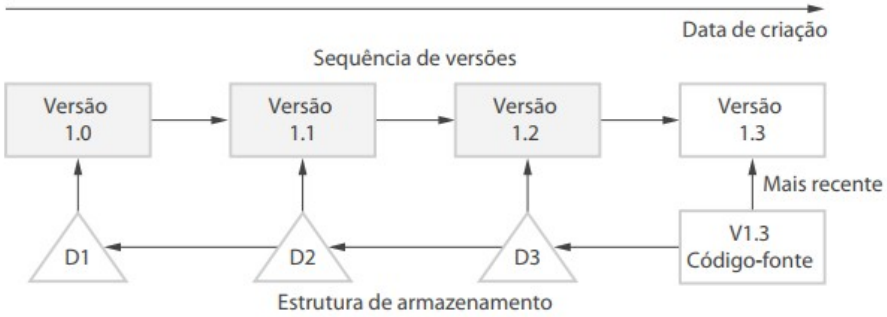
Na Figura 25.3, você pode ver que diferentes baselines usam versões diferentes dos componentes de cada codeline

Figura 25.3 *Codelines e baselines*



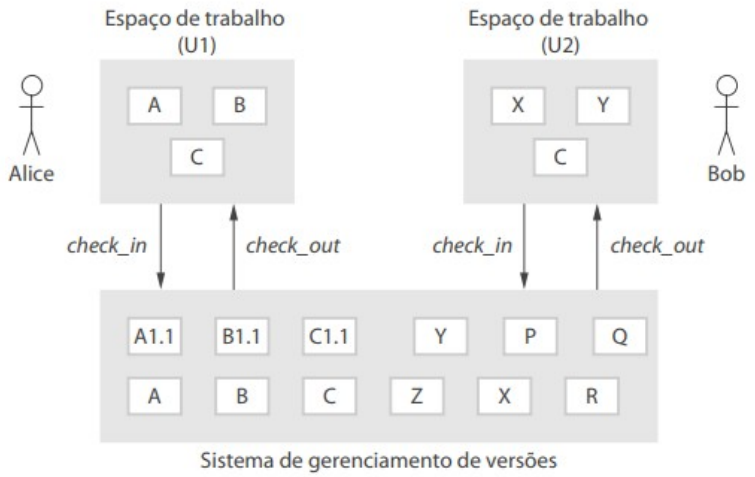
Quando uma nova versão é criada, o sistema simplesmente armazena um delta (uma lista de diferenças) entre a nova versão e a mais antiga, usada para criar essa nova versão (mostrado na parte inferior da Figura 25.4).

Figura 25.4 Gerenciamento de armazenamento usando deltas



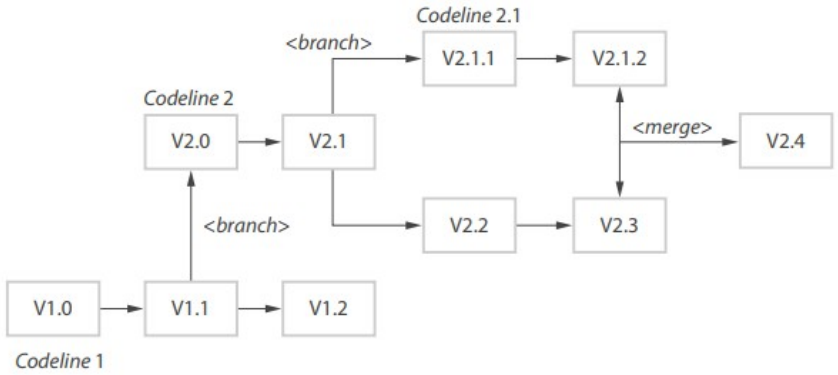
Quando as mudanças forem concluídas, eles realizam o check-in de componentes para o repositório. Esse processo é ilustrado na Figura 25.5

Figura 25.5 *Check-in* e *check-out* a partir de um repositório de versões



Uma consequência do desenvolvimento independente do mesmo componente é que codelines podem se ramificar. Em vez de uma sequência linear de versões que refletem as mudanças para o componente ao longo do tempo, pode haver várias sequências independentes, como mostrado na Figura 25.6.

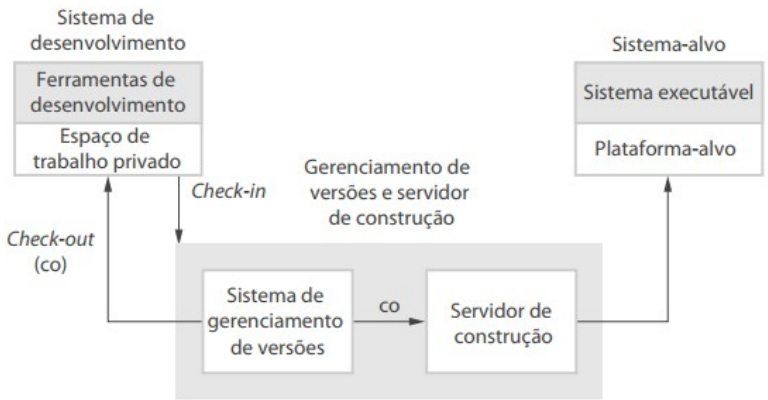
Figura 25.6 *Branching e merging*



25.3 Construção de sistemas

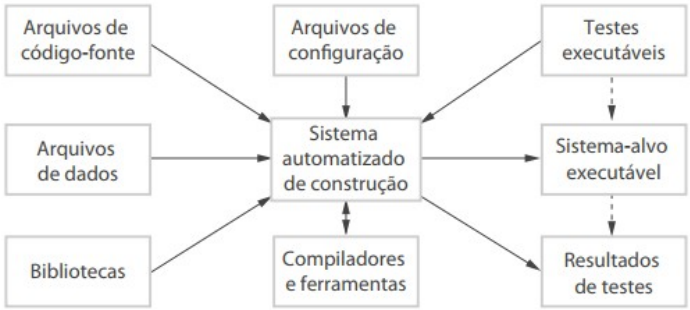
A construção é um processo complexo e potencialmente sujeito a erros, pois pode haver três diferentes plataformas de sistemas envolvidas (Figura 25.7):

Figura 25.7 Plataformas de desenvolvimento, construção e alvo



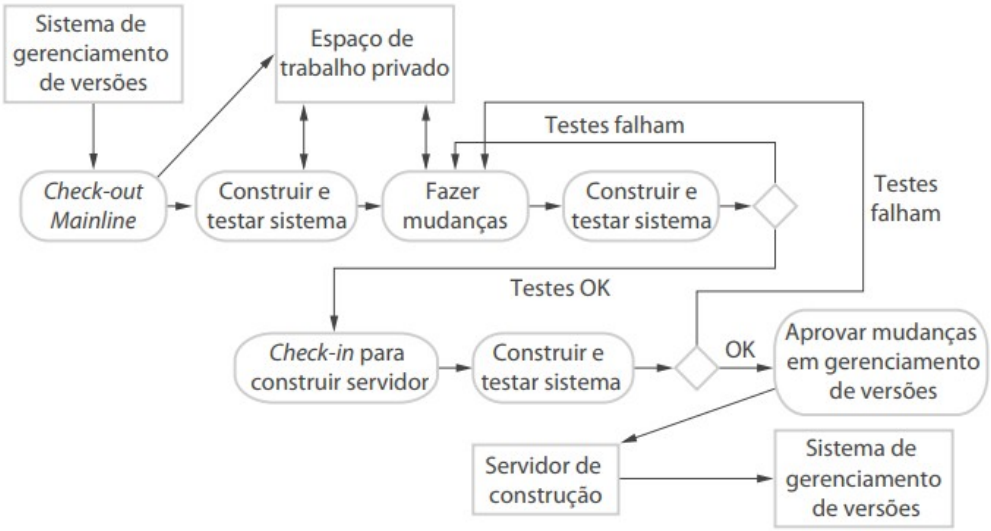
sempre faz sentido usar uma ferramenta de construção automatizada para criar uma construção de sistema (Figura 25.8).

Figura 25.8 A construção de sistemas



As construções frequentes podem ser parte de um processo de integração contínua, como mostrado na Figura 25.9

Figura 25.9 Integração contínua



25.4 Gerenciamento de releases

Um release de sistema é uma versão de um sistema de software distribuída aos clientes. Para softwares de mercado de massa, é normalmente possível identificar dois tipos de releases, chamados releases principais, que fornecem nova e significativa funcionalidade, e releases menores, que reparam bugs e corrigem problemas de clientes que foram relatados.

Um release de sistema não é apenas o código executável do sistema. Ele também pode incluir:

- arquivos de configuração definindo como o release deve ser configurado para instalações particulares;
- arquivos de dados, tais como arquivos de mensagens de erro, que são necessários para uma operação bem--sucedida do sistema;
- um programa de instalação, o qual é usado para ajudar a instalar o sistema no hardware-alvo;
- documentação eletrônica e em papel, que descrevem o sistema;
- empacotamento e publicidade associada, projetadas para esse release.

Capítulo 26 : Melhorias de processos

Existe uma procura constante da indústria por softwares mais baratos e melhores, os quais precisam ser entregues em deadlines cada vez mais rigorosos.

Consequentemente, muitas empresas de software voltaram-se para a melhoria de processos de software como uma forma de melhorar a qualidade de seu software, reduzindo custos ou acelerando seus processos de desenvolvimento.

A melhoria de processos implica a compreensão dos processos existentes e sua mudança para aumentar a qualidade de produtos e/ou reduzir custos e o tempo de desenvolvimento.

São usadas duas abordagens bastante diferentes para a melhoria e a mudança de processos:

1. A abordagem de maturidade de processo, a qual se centra em melhorar o gerenciamento de processos e projetos e em introduzir boas práticas de engenharia de software em uma organização. O nível de maturidade de processos reflete o grau em que as boas práticas técnicas e de gerenciamento foram adotadas nos processos de desenvolvimento de software da organização. Os principais objetivos dessa abordagem são produtos de melhor qualidade e

previsibilidade de processo.

2. A abordagem ágil, a qual se centra no desenvolvimento iterativo e na redução de overheads gerais no processo de software. As principais características dos métodos ágeis são a entrega rápida de funcionalidade e a capacidade de resposta às mudanças de requisitos de cliente.

Geralmente, os adeptos de uma dessas abordagens são céticos em relação aos benefícios da outra.

A abordagem de maturidade de processo está enraizada no desenvolvimento dirigido a planos e em geral requer aumento de 'overhead', no sentido de que são introduzidas atividades que não são diretamente relevantes para a programação.

As abordagens ágeis concentram-se no código que está sendo desenvolvido e minimizam, deliberadamente, a formalidade e a documentação

Deming (e outros) introduziram a ideia de controle estatístico de qualidade, a qual se baseia em medir o número de defeitos de produto e relacionar esses defeitos ao processo. O objetivo é reduzir o número de defeitos de produto, analisar e modificar o processo para que as chances de introdução de defeitos sejam reduzidas e a detecção de defeitos seja melhorada. Uma vez que se tenha conseguido uma baixa contagem de defeitos, o processo é padronizado e, em seguida, um novo ciclo de melhorias tem início.

Humphrey (1988)...

...

W. E. Deming, em seu trabalho com a indústria japonesa após a Segunda Guerra Mundial, aplicou na indústria seus conceitos do controle estatístico de processos. Embora existam diferenças importantes, esses conceitos são tão aplicáveis ao software quanto para automóveis, máquinas fotográficas, relógios de pulso e aço.

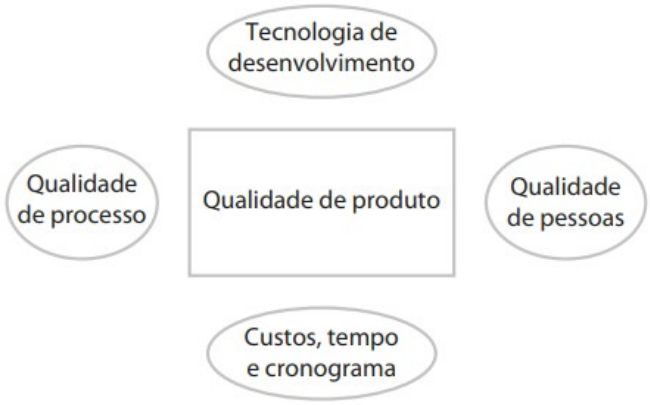
Porém,....

Esse relacionamento entre qualidade de produto e qualidade de processo é menos óbvio quando o produto é intangível e dependente, em certa medida, de processos intelectuais que não podem ser automatizados.

A qualidade de software não é influenciada por seu processo de manufatura, mas por seu processo de projeto, no qual as habilidades e a experiência das pessoas são significativas.

Para produtos de software, ou quaisquer outros produtos intelectuais, tais como livros ou filmes em que a qualidade do produto depende de seu projeto, existem quatro importantes fatores que afetam a qualidade de produto.

Figura 26.1 Fatores que afetam o produto de software



26.1 O processo de melhoria de processos

Não existe algo como um processo de software 'ideal' ou 'padrão' que seja aplicável a todas as organizações ou para todos os produtos de software de um tipo particular. Cada empresa precisa desenvolver seu próprio processo dependendo de seu tamanho, de seu background e das habilidades de seu pessoal, do tipo de software que esteja sendo desenvolvido, do cliente e dos requisitos de mercado, bem como da cultura da empresa.

Portanto, a melhoria dos processos não significa apenas a adoção de métodos particulares ou ferramentas ou o uso de um processo genérico publicado.

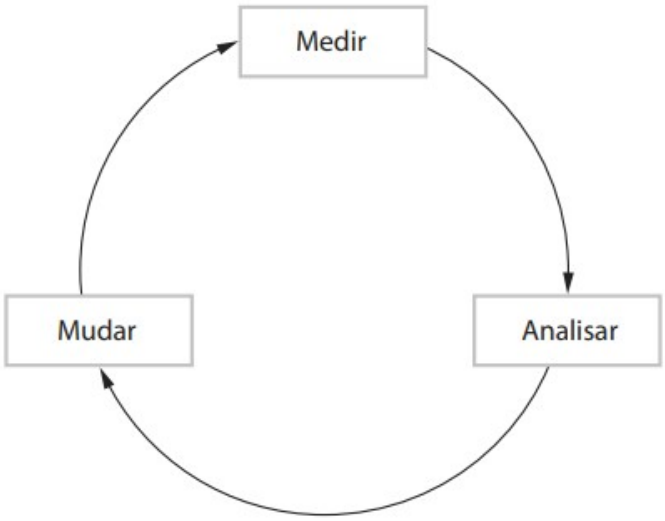
… deve considerar quais aspectos do processo você deseja melhorar. Seu objetivo pode ser a melhoria da qualidade de software e, assim, você pode querer introduzir novas atividades de processo que alterem a forma como o software é desenvolvido e testado. Você pode estar interessado em melhorar algum atributo do processo em si e precisa decidir quais atributos de processo são mais importantes para sua empresa.

Tabela 26.1 Atributos de processo

Características de processo	Questões principais
Compreensibilidade	Em que medida o processo é definido explicitamente e quão fácil é entender sua definição?
Padronização	Em que medida o processo é baseado em um processo genérico padrão? Isso pode ser importante para alguns clientes que exigem conformidade com um conjunto definido de padrões de processos. Em que medida o mesmo processo é usado em todas as partes de uma empresa?
Visibilidade	As atividades do processo culminam em resultados claros, de modo que o progresso do processo seja visível externamente?
Capacidade de medição	O processo inclui a coleta de dados ou outras atividades que permitam que as características do produto ou processo sejam medidas?
Capacidade de apoio	Em que medida as ferramentas de software podem ser usadas para apoiar as atividades de processo?
Aceitabilidade	O processo definido é aceitável e usável pelos engenheiros responsáveis pela produção do produto de software?
Confiabilidade	O processo é projetado de forma que erros de processo sejam evitados ou encontrados antes de resultarem em erros de produto?
Robustez	O processo pode continuar apesar de problemas inesperados?
Manutenibilidade	O processo pode evoluir para refletir a mudança de requisitos organizacionais ou melhorias de processos identificados?
Rapidez	Quão rápido pode ser concluído o processo de entrega de um sistema após a conclusão de uma determinada especificação?

O processo de melhoria de processos é um processo cíclico, conforme mostrado na Figura 26.2. Ele envolve três subprocessos:

Figura 26.2 O ciclo de melhorias de processos



26.2 Medição de processos

As medições de processos são dados quantitativos sobre o processo de software, tal como o tempo necessário para realizar alguma atividade do processo.

As medições de processo podem ser usadas para avaliar a melhoria da eficiência de um processo.

Três tipos de métricas de processo podem ser coletados:

1. O tempo necessário para um processo específico ser concluído. Exemplo: Reuniões com o cliente para compreender os requisitos do sistema.

2. Os recursos necessários para um determinado processo. Exemplo: Transporte e hospedagem para o Engenheiro de Software.

3. O número de ocorrências de um determinado evento. Exemplo: quantidade de alterações nos requisitos, após aprovação.

Uma dificuldade fundamental no processo de medição é saber quais informações sobre o processo devem ser coletadas para apoiar a melhoria de processo.

O paradigma GQM (Metas-questões-métricas, do inglês Goal-Question-Metric) (Figura 26.3) é usado na melhoria de processos para ajudar a responder três questões críticas:

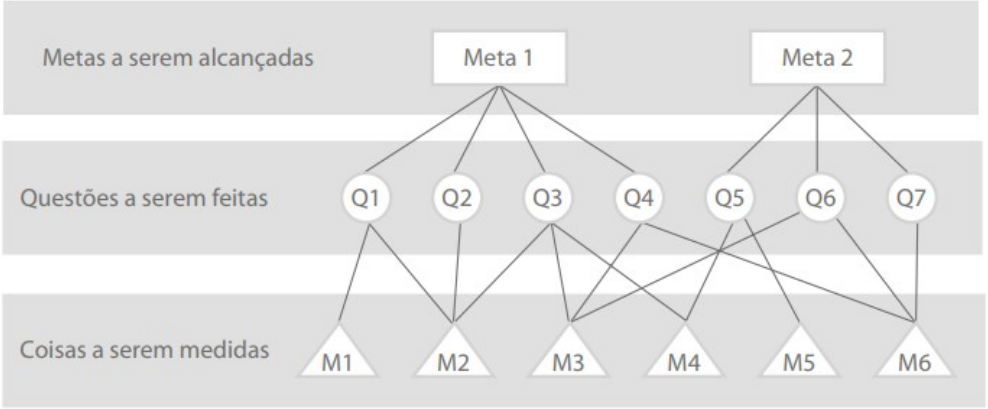
1. Por que estamos introduzindo melhorias de processos?

2. De quais informações precisamos para ajudar a identificar e avaliar as melhorias?

3. Quais medições de processo e produto são necessárias para fornecer essas informações

Essas questões estão diretamente relacionadas com as abstrações (metas, questões, métricas) do paradigma GQM.

Figura 26.3 O paradigma GQM



Atividade Prática Individual sobre GQM

1. Escolha de um Processo ou Produto de Software

Selecionar um processo e um produto de software que sirva de objeto para análise.

Faça um parágrafo apresentando sua escolha.

2. Definição do Objetivo (Goal)

Descrever de forma clara e objetiva a meta de melhoria ou avaliação desejada para o processo e uma para o produto escolhido (duas metas). Para facilitar, pode usar a seguinte estrutura:

- Analisar [objeto]
- Para o propósito de [melhoria, controle, avaliação]
- Com foco em [aspecto de qualidade: eficiência, eficácia, qualidade, etc.]
- Do ponto de vista de [usuário, desenvolvedor, gestor]
- No contexto de [situação/turma/empresa específica].

3. Formulação das Questões (Questions)

Elabore, no mínimo, duas perguntas que ajudem a verificar se as metas estão sendo alcançadas.

4. Definição das Métricas (Metrics)

Para cada pergunta, indique um ou mais indicadores quantificáveis que possam ser coletados de forma objetiva, relacionando-os diretamente às questões formuladas.

5. Esquematização da Coleta e Interpretação

Explicite como coletaria esses dados (fontes, frequência, instrumentos) e a descreva rapidamente como interpretaria os resultados obtidos.

26.3 Análise de processos

A análise de processo é o estudo dos processos para ajudar a compreender suas principais características e como esses processos são executados na prática, pelas pessoas envolvidas.

A análise de processos tem vários objetivos estreitamente relacionados:

1. Entender as atividades envolvidas no processo e os relacionamentos entre essas atividades.

2. Entender os relacionamentos entre as atividades de processo e as medições feitas.

3. Relacionar o processo específico ou processos que você está analisando com processos comparáveis localizados em outro lugar na organização, ou com processos idealizados de mesmo tipo.

As técnicas mais usadas na análise de processos são:

1. Questionários e entrevistas

2. Estudos etnográficos.

Tabela 26.2 Aspectos da análise de processos

Aspecto de processo	Questões
Adoção e padronização	O processo está documentado e padronizado em toda a organização? Se não, isso significa que quaisquer medições efetuadas são específicas apenas para uma única instância de processo? Se os processos não forem padronizados, então as mudanças para um processo podem não ser transferíveis para processos comparáveis, em outros lugares da empresa.
Prática de engenharia de software	Existem boas práticas de engenharia de software, que não estão incluídas no processo? Por que elas não estão incluídas? A falta dessas práticas afeta as características de produto, tal como o número de defeitos em um sistema de software entregue?
Restrições organizacionais	Quais são as restrições organizacionais que afetam o projeto de processo e as maneiras como o processo é executado? Por exemplo, se o processo envolve lidar com material confidencial, pode haver atividades no processo para verificar se as informações confidenciais não estão incluídas em qualquer material que será liberado para organizações externas. Restrições organizacionais podem significar que possíveis mudanças de processo não podem ser realizadas.
Comunicações	Como as comunicações são gerenciadas no processo? Como os problemas de comunicação se relacionam com as medições de processo feitas? Os problemas de comunicação são uma questão importante em muitos processos e, muitas vezes, os gargalos de comunicação são motivo para atrasos no projeto.

Ao se analisar um processo, costuma ser útil iniciar com um modelo de processo que defina as atividades no processo e as entradas e saídas dessas atividades.

O modelo também pode incluir informações sobre os atores de processo — as pessoas ou os papéis responsáveis pela execução das atividades e os resultados essenciais que devem ser apresentados.

EXEMPLO

Os workflows representam modelos de processo de negócios e são geralmente representados por meio de uma notação gráfica, como diagramas de atividades da UML ou BPMN, notação de modelagem de processo de negócio

Figura 19.8 *Workflow* do pacote de férias

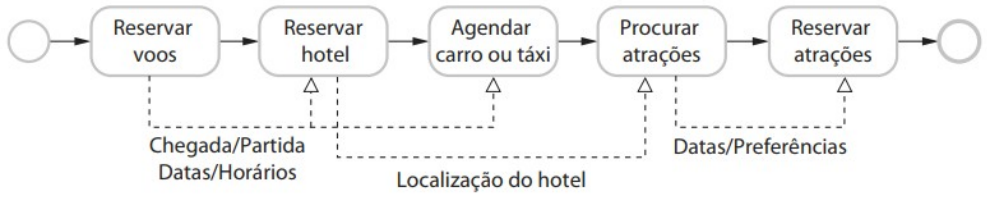
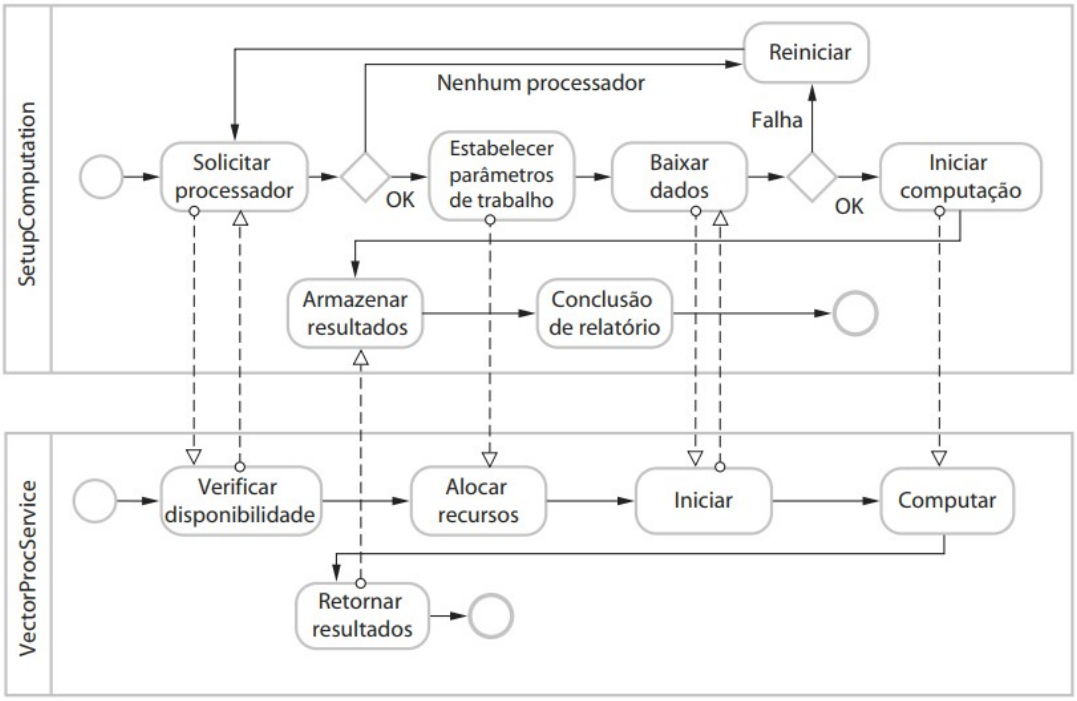


Figura 19.11 Interação de *workflows*



Muitas vezes, a discussão com as pessoas envolvidas no processo e as observações desse processo são estruturadas em torno de um conjunto de questões sobre o modelo de processo formal. Exemplos dessas questões podem ser:

1. Quais atividades ocorrem na prática, mas não são mostradas no modelo?

2. Existem atividades de processos, mostradas no modelo, que você (o ator de processo) acredita serem ineficientes?

3. O que acontece quando as coisas dão errado?

4. Quem são os atores envolvidos nos diferentes estágios do processo e como eles se comunicam?

5. Qual suporte de ferramentas é usado para as atividades mostradas no modelo?

Ao concluir uma análise do processo de software, você deve ter uma compreensão mais profunda desse processo e do potencial de melhoria de processos no futuro. Você também deve compreender as restrições nas melhorias de processos e como estas podem limitar o escopo de melhorias que podem ser introduzidas.

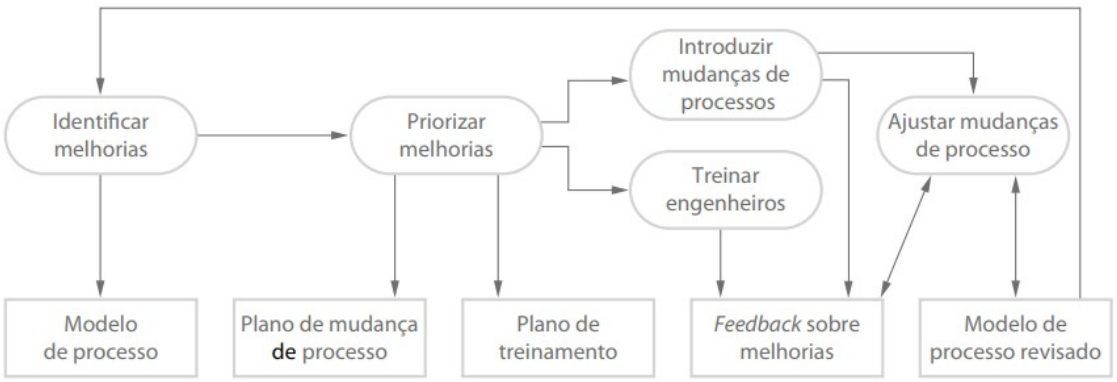
26.4 Mudança de processos

As mudanças de processo envolvem fazer modificações no processo existente.

Pode fazer isso por meio da introdução de novas práticas, métodos ou ferramentas; mudando a ordem das atividades do processo; introduzindo ou removendo os entregáveis do processo; melhorando as comunicações; ou introduzindo novos papéis e responsabilidades.

Existem cinco estágios principais no processo de mudança de processos (Figura 26.4)

Figura 26.4 O processo de mudança de processos



Geralmente, é imprudente introduzir muitas mudanças ao mesmo tempo. Além das dificuldades de treinamento, a introdução de muitas mudanças impossibilita a avaliação do efeito de cada uma no processo.

Duas grandes dificuldades que os envolvidos nos processos de mudança podem enfrentar:

1. Resistência às mudanças.

2. Persistência de mudanças.

26.5 Framework CMMI de melhorias de processos

O Instituto de Engenharia de Software (SEI, do inglês Software Engineering Institute) dos Estados Unidos foi criado visando a melhoria das capacidades da indústria norte-americana de software. Em meados de 1980, o SEI iniciou um estudo sobre como avaliar as capacidades dos prestadores de serviços de software. O resultado dessa avaliação de capacidade foi o Modelo Maturidade e de Capacidade (CMM, do inglês Capability Maturity Model) do SEI (PAULK et al., 1993; PAULK et al., 1995).

O modelo CMMI é muito complexo, com mais de mil páginas de descrição. Para nossa discussão, eu simplifiquei radicalmente esse modelo. Os principais componentes do modelo são:

1. Um conjunto de áreas de processo relacionadas às atividades de processos de software.

2. Um número de metas, que são descrições abstratas de um estado desejável a ser atingido por uma organização.

3. Um conjunto de boas práticas, que são as descrições das formas de como alcançar uma meta.

Tabela 26.3 Áreas de processo no CMMI

Categoria	Área de processo
Gerenciamento de processos	Definição de processo organizacional (OPD, do inglês *organizational process definition*) Foco de processo organizacional (OPF, do inglês *organizational process focus*) Treinamento organizacional (OT, do inglês *organizational training*) Desempenho de processo organizacional (OPP, do inglês *organizational process performance*) Inovação e implantação organizacional (OID, do inglês *organizational innovationn and deployment*)
Gerenciamento de projetos	Planejamento de projeto (PP, do inglês *project planning*) Monitoração e controle de projeto (PMC, do inglês *project monitoring and control*) Gerenciamento de acordo com fornecedores (SAM, do inglês *supplier agreement management*) Gerenciamento de projeto integrado (IPM, do inglês *integrated project management*) Gerenciamento de riscos (RSKM, do inglês *risk management*) Gerenciamento quantitativo de projeto (QPM, do inglês *quantitative project management*)
Engenharia	Gerenciamento de requisitos (REQM, do inglês *requirements management*) Desenvolvimento de requisitos (RD, do inglês *requirements development*) Solução técnica (TS, do inglês *technical solution*) Integração de produto (PI, do inglês *product integration*) Verificação (VER, do inglês *verification*) Validação (VAL, do inglês *validation*)
Suporte	Gerenciamento de configuração (CM, do inglês *configuration management*) Garantia de qualidade de processo e produto (PPQA, do inglês *process and product quality management*) Medição e análise (MA, do inglês *measurement and analysis*) Análise de decisão e resolução (DAR, do inglês *decision analysis and resolution*) Análise causal e resolução (CAR, do inglês *causal analysis and resolution*)

Tabela 26.4 Áreas de processo no CMMI

Meta	Área de processo
Ações corretivas são gerenciadas até a conclusão, quando o desempenho ou os resultados do projeto se desviam significativamente do plano.	Monitoração e controle de projeto (meta específica)
O desempenho real e o progresso do projeto são monitorados contra o plano de projeto.	Monitoração e controle de projeto (meta específica)
Os requisitos são analisados e validados, e uma definição da funcionalidade requerida é desenvolvida.	Desenvolvimento de requisitos (meta específica)
Causas-raiz de defeitos e outros problemas são sistematicamente determinados.	Análise causal e resolução (meta específica)
O processo é institucionalizado como um processo definido.	Meta genérica

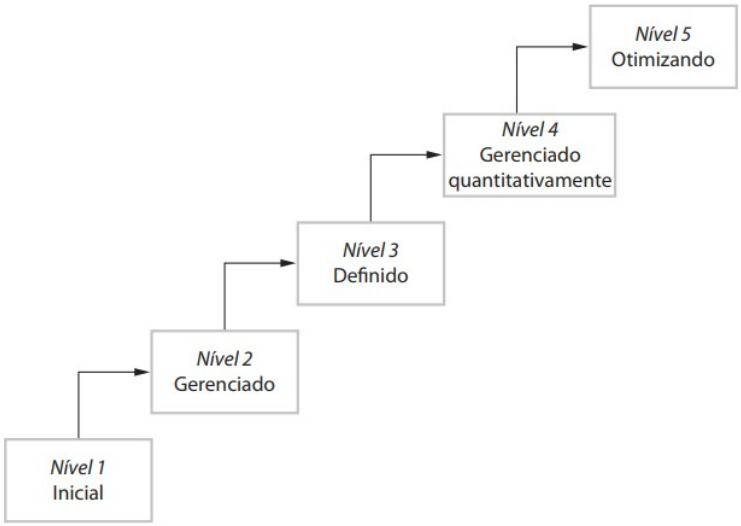
Uma avaliação de CMMI envolve examinar os processos em uma organização e classificar esses processos ou áreas de processo em uma escala de seis pontos relacionada ao nível de maturidade em cada área de processo.

A escala de seis pontos atribui níveis de maturidade para uma área de processo, da seguinte forma, Figura 26.5.

26.5.1 O modelo CMMI por estágios

Modelos por estágio do CMMI é usado para avaliar a capacidade da organização como um todo

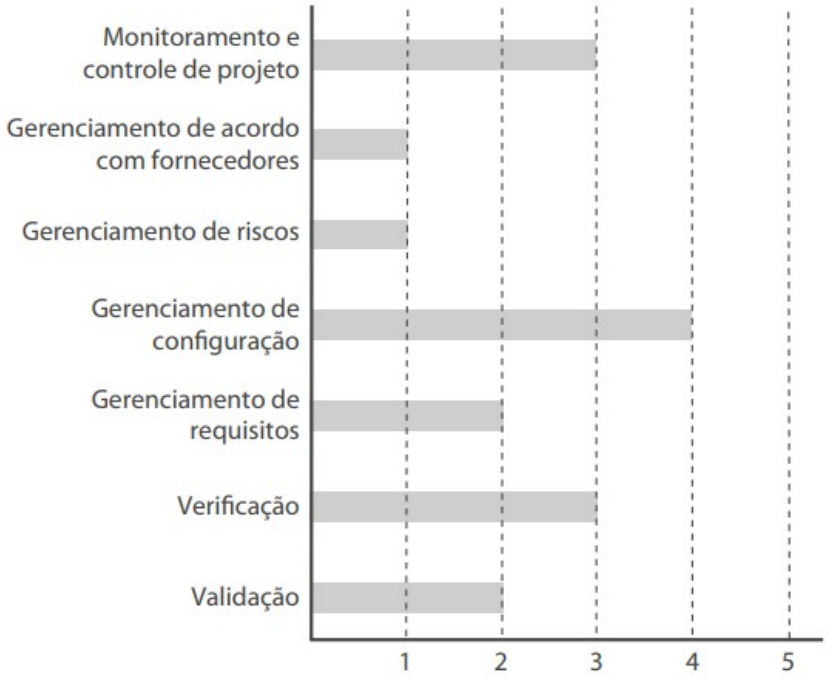
Figura 26.5 O modelo de maturidade CMMI por estágios



26.5.2 O modelo CMMI contínuo

são modelos mais refinados que consideram práticas individuais ou grupos de práticas e avaliam o uso de boas práticas dentro de cada grupo de processo.

Figura 26.6 Um perfil de capacidade de processos



Mini resumos

Capítulo 24: Gerenciamento de Qualidade

1. Qualidade de Software

A qualidade de software é subjetiva, não sendo diretamente comparável à qualidade na manufatura111. Isso acontece porque é difícil escrever especificações de software completas e precisas, e as características de qualidade, como a manutenibilidade, não podem ser medidas de forma não ambígua2222. O gerenciamento da qualidade do software envolve o julgamento da equipe para decidir se um nível aceitável de qualidade foi alcançado3. O software de qualidade não se trata apenas de funcionalidade, mas também de atributos não funcionais4.

Alguns atributos de qualidade importantes, segundo Boehm et al. (1978), incluem segurança, confiabilidade, usabilidade e eficiência5555.

2. Padrões de Software

Os padrões de software são cruciais para o gerenciamento de qualidade6. Eles oferecem um "framework" para definir o que significa "qualidade" para uma organização7. Existem dois tipos principais de padrões:

- **Padrões de Produto:** Aplicam-se ao software que está sendo desenvolvido. Incluem padrões de documentos, de documentação e de codificação8888.
- **Padrões de Processo:** Definem como o processo de desenvolvimento deve ser seguido9999. Devem encapsular boas práticas de desenvolvimento10.

3. Revisões e Inspeções

Revisões e inspeções são atividades para verificar a qualidade dos "entregáveis" do projeto, como o software e a documentação11. O processo de revisão envolve três fases: atividades pré-revisão, a reunião de revisão e atividades pós-revisão12. As inspeções de programa são revisões em pares focadas em encontrar bugs e não exigem que o programa seja executado13.

4. Medições e Métricas de Software

A medição de software é o processo de atribuir um valor numérico a um atributo de um componente, sistema ou processo14. Uma

métrica é uma característica que pode ser objetivamente medida15. Exemplos de métricas incluem o tamanho do código em linhas, o número de defeitos relatados ou o tempo necessário para o desenvolvimento16161616.

As métricas podem ser de

controle (focadas no processo, como o tempo para reparar defeitos) ou de **previsão** (focadas no produto, como a complexidade ciclomática)17.

Capítulo 25: Gerenciamento de Configuração

O Gerenciamento de Configuração (CM) lida com políticas, processos e ferramentas para gerenciar as mudanças nos sistemas de software18. É essencial para evitar perder o controle de quais mudanças e versões de componentes foram incorporadas19191919.

As atividades do CM incluem:

- **Gerenciamento de Mudanças:** Lida com propostas de mudança e correções de defeitos20202020. O processo envolve submeter, verificar, registrar, analisar o impacto e, finalmente, modificar e testar o software21.

- **Gerenciamento de Versões:** Envolve o uso de "codelines" e "baselines" para rastrear diferentes versões de componentes22. O sistema pode armazenar apenas as diferenças (

  deltas) entre as versões para economizar espaço23. O processo de

  check-in e *check-out* permite que desenvolvedores trabalhem em suas cópias privadas e submetam as mudanças para um repositório central24242424.
- **Construção e Integração de Sistemas:** A construção de um sistema pode ser um processo complexo, e é recomendado o uso de ferramentas automatizadas para criá-la25252525. As construções frequentes podem fazer parte de um processo de

  integração contínua, em que as mudanças são testadas e integradas regularmente26262626.

Capítulo 26: Melhoria de Processos

A melhoria de processos é um esforço para entender os processos existentes e mudá-los para aumentar a qualidade ou reduzir custos e tempo de desenvolvimento27. É um processo cíclico que envolve

análise, **medição** e **mudança**28282828.

1. O Paradigma GQM

O paradigma

GQM (Goals-Questions-Metrics) é uma ferramenta usada para auxiliar na melhoria de processos29.

- **Metas (Goals):** Definem por que você está introduzindo melhorias de processo30.
- **Questões (Questions):** Ajudam a identificar as informações necessárias para avaliar as melhorias31.
- **Métricas (Metrics):** São as medições de processo e produto necessárias para fornecer essas informações32.

2. Análise de Processos

A análise de processo é o estudo dos processos para entender suas características e como são executados na prática33. Ela busca entender as atividades e as relações entre elas, as medições feitas, e comparar os processos com outros34.

Alguns aspectos importantes da análise de processo incluem:

- **Adoção e Padronização:** Verificar se os processos estão documentados e padronizados em toda a organização35.
- **Práticas de Engenharia de Software:** Analisar se boas práticas estão sendo incluídas no processo36.
- **Restrições Organizacionais:** Entender as restrições que podem afetar o projeto do processo37.
- **Comunicações:** Avaliar como a comunicação é gerenciada e se os problemas de comunicação estão causando atrasos38.

Espero que este resumo ajude você a se preparar para a prova! Se tiver qualquer dúvida sobre um tópico específico, é só me dizer.